



SDN using EVE

TSM - Advanced Communication Architecture

Dimitri Julmy

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://github.com/HezelTm/tsm_advcomarc
Creation date	2026-05-21
Submission date	2026-05-27

Introduction

Ce travail pratique s'inscrit dans le cours *TSM – Advanced Communication Architecture* et porte sur la mise en place d'un environnement de simulation réseau virtuel. L'outil central de ce laboratoire est *EVE-NG*, une plateforme permettant de créer et d'exécuter des topologies réseau entièrement dans un navigateur web, sans matériel physique. Nous commencerons par installer et configurer cet environnement, puis nous explorerons ses fonctionnalités et le comparerons à des alternatives existantes. Une fois l'environnement opérationnel, nous déploierons une topologie réseau composée de plusieurs routeurs et hôtes virtuels interconnectés. L'ensemble de ce déploiement sera automatisé au moyen de scripts, afin d'illustrer les principes de gestion programmatique des infrastructures réseau.

Dans un second temps, nous configurerons le routage entre les différents segments du réseau pour permettre aux hôtes de communiquer entre eux, quelle que soit leur position dans la topologie. Des tests de connectivité seront ensuite effectués pour valider le bon fonctionnement de l'infrastructure déployée. Nous documenterons également les difficultés rencontrées lors de la mise en œuvre ainsi que les solutions envisagées. Ce rapport présente l'ensemble de la démarche suivie, depuis l'installation de l'environnement jusqu'aux résultats des tests finaux. Il constitue ainsi une trace complète du travail réalisé et des enseignements tirés de cette expérience.

Implementation

Mise en place EVE

La procédure ci-dessous est détaillée pour un environnement *Arch Linux*.

Installer *VirtualBox* (<https://wiki.archlinux.org/title/VirtualBox>):

```
1 yay -S virtualbox
```

Télécharger *Free EVE Community Edition Version 6.2.0-4*.

Créer une nouvelle VM (machine > New, Ctrl + n) avec les paramètres suivants, puis cliquer sur Finish :

- *Virtual machine name and operating system* :
 - VM Name : EVE-NG
 - ISO Image : /path/to/image/eve-ce-prod-6.2.0-4-full.iso
 - OS : Linux
- *Specify virtual hardware*
 - Base Memory : 8000 MB
 - CPU : 4
- *Specify virtual hard disk*
 - Disk Size : 50 GB

Dans Settings > Network (mode Expert), passer l'Adapter 1 en Bridged Adapter. Laisser les autres paramètres par défaut.

Au premier démarrage, suivre l'installation de EVE-NG Community 6.2.0-4 (langue et clavier). Une fois terminée, la VM retourne sur GRUB : retirer le disque via Device > Optical Device > Remove Disk From Virtual Device, puis éteindre la machine (Power off the machine). Au redémarrage, se connecter avec :

- Nom d'utilisateur : root
- Mot de passe : eve

Conserver les valeurs par défaut pour les configurations suivantes. La machine redémarre ; se reconnecter avec les mêmes identifiants. L'URL de l'interface web est affichée dans la console (<http://192.168.1.113> dans notre cas). Se connecter avec les identifiants suivant :

- Nom d'utilisateur : admin
- Mot de passe : eve

Télécharger l'image du routeur *Arista vEOS-lab-4.36.0.1F.qcow2* et le *bootloader* *Aboot-veos-serial-8.0.2.iso* depuis le support de cours Moodle.

Avant de démarrer la VM, activer la virtualisation imbriquée (*nested virtualization*) dans *VirtualBox* pour que KVM soit disponible dans le guest (VM éteinte) :

```
1 # Sur la machine hôte (VM éteinte)
2 VBoxManage modifyvm "EVE-NG" --nested-hw-virt on
```

Démarrer la VM, puis transférer les images et corriger les permissions :

```
1 # Sur la machine hôte
2 ssh root@10.24.45.108 "mkdir -p /opt/unetlab/addons/qemu/veos-4.36.0.1F"
3 scp ~/Downloads/vEOS-lab-4.36.0.1F.qcow2 root@10.24.45.108:/opt/unetlab/addons/qemu/
veos-4.36.0.1F/hda.qcow2
4 scp ~/Downloads/Abboot-veos-serial-8.0.2.iso root@10.24.45.108:/opt/unetlab/addons/qemu/
veos-4.36.0.1F/cdrom.iso
5
6 # Sur la VM
7 echo "kvm" > /opt/unetlab/platform
8 /opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

Source : Youtube - TechNPlay - How to Install EVE-NG on Oracle VirtualBox 2025

EVE and alternate solutions

While installing EVE, please look for the differences between EVE and another alternate solution: GNS3, and document the findings.

GNS3 :

- Émulateur réseau *open-source*.
- Exécute de vraies images *IOS* (pas de logiciels simulés).
- Le comportement des équipements est identique au matériel physique (grâce aux vraies images *IOS*).
- Architecture client-serveur (l'interface graphique tourne localement tandis que le *backend* d'émulation tourne localement ou sur un serveur distant).

+ Avantages :

- Plus grande communauté (davantage de tutoriels, *templates* et ressources)
- Expérience bureau native
- Intégration officielle *Cisco*
- Excellente intégration *Wireshark*
- Développement actif

- Désavantages :

- Architecture mono-utilisateur (non conçue pour les simulations partagées)
- La configuration de la VM GNS3 peut être complexe
- Émulation *IOS legacy* uniquement pour les images anciennes

EVE :

- Plateforme d'émulation réseau fonctionnant entièrement comme une application serveur *Linux*.
- Accessible via un navigateur web (aucune installation locale requise).
- Conçue pour les environnements de labs partagés (plusieurs utilisateurs peuvent se connecter simultanément au même serveur *EVE-NG*, chacun travaillant dans sa propre topologie).
- Deux éditions : *Community* (gratuite) et *Professional* (payante)

+ Avantages :

- Accès via navigateur
- Environnements de labs partagés
- Fort support multi-constructeurs
- Déduplication mémoire *UKSM* (exécuter 10 routeurs identiques consomme beaucoup moins de RAM que 10 processus séparés)

- Désavantages :

- Configuration d'un serveur requise
- Les meilleures fonctionnalités sont payantes
- Communauté plus petite que *GNS3*

Comparaison des fonctionnalités

Fonctionnalité	GNS3	EVE-NG Community	EVE-NG Pro
Coût	Gratuit	Gratuit	\$120/an
Interface	Interface bureau (<i>Windows/Mac/Linux</i>)	Navigateur web	Navigateur web
Installation	Application locale + VM <i>GNS3</i>	Serveur <i>Linux</i> (<i>bare metal</i> ou VM)	Serveur <i>Linux</i>
Support multi-utilisateurs	✗ Mono-utilisateur	✗ Limité	☑ Multi-utilisateurs complet
Support <i>Cisco IOS</i>	☑ Complet	☑ Complet	☑ Complet
<i>Cisco IOS-XE/XR/NX-OS</i>	☑	☑	☑
<i>Palo Alto / Fortinet / Juniper</i>	☑ Via <i>QEMU</i>	☑ Via <i>QEMU</i>	☑ Via <i>QEMU</i>
Efficacité RAM	Standard	Standard	☑ Déduplication <i>UKSM</i>
Support conteneurs <i>Docker</i>	☑	☑	☑
Capture de paquets intégrée	☑ Intégration <i>Wireshark</i>	☑	☑
Communauté et <i>templates</i>	Très grande	Grande	Grande + officielle
Labs certification <i>Cisco</i>	☑ <i>GNS3 Academy</i> officielle	Labs communautaires	Labs <i>EVE-NG</i> officiels

Source : [raghededris.com](https://www.raghededris.com/2026/03/13/eve-ng-vs-gns3-comparison/) - EVE-NG vs GNS3 : <https://www.raghededris.com/2026/03/13/eve-ng-vs-gns3-comparison/>

Analyse de l'environnement EVE

Take a look at the tools and configurations made available by the program.

EVE-NG (*Emulated Virtual Environment*) expose une interface web permettant de concevoir, démarrer et superviser des topologies réseau entièrement depuis un navigateur.

Outils principaux :

- **Topology Editor** : éditeur graphique *drag-and-drop* pour créer des topologies réseau. Permet d'ajouter des nœuds, de les relier par des liens virtuels et d'exporter/importer des labs au format `.unl`.
- **Console d'accès** : accès aux nœuds via *Telnet*, *SSH* ou *VNC* selon le type d'équipement (*IOL*, *QEMU*, *Docker*).
- **Capture de paquets** : capture intégrée sur n'importe quel lien de la topologie, avec ouverture directe dans *Wireshark* sur la machine cliente.
- **Gestion des labs** : création, sauvegarde et partage de labs sous forme d'archives *ZIP*.

Configurations disponibles :

- **Templates de nœuds** : modèles préconfigurés pour une multitude de constructeurs (*Cisco*, *Juniper*, *Arista*, *Palo Alto*, *Fortinet*, etc.). Chaque template définit le type d'hyperviseur, le nombre d'interfaces, la *RAM* et le *CPU* alloués.
- **Types de réseaux** : réseau de management (*Cloud0*), bridges internes (*Net*) et connexions vers le réseau hôte (*Cloud1-Cloud9*).
- **Configurations de démarrage** : possibilité de pré-charger une configuration initiale sur chaque nœud avant le démarrage du lab.
- **Gestion des utilisateurs** : comptes utilisateurs avec rôles (administrateur, étudiant) et accès simultanés aux labs en édition *Pro*.

Source : [eve-ng.net](https://www.eve-ng.net/index.php/documentation/) - Documentation officielle EVE-NG : <https://www.eve-ng.net/index.php/documentation/>

Please check the infrastructure and librairies made available by EVE.

EVE-NG s'appuie sur un socle *Linux* (*Ubuntu Server 22.04 LTS*) et tire parti de plusieurs technologies de virtualisation pour émuler des équipements réseau hétérogènes.

Infrastructure :

- **KVM/QEMU** : hyperviseur principal utilisé pour exécuter les images *QEMU* des équipements réseau (routeurs, *firewalls*, *switches*). Chaque nœud tourne dans une *VM* légère.
- **IOL (IOS on Linux)** : permet de faire tourner certaines images *Cisco IOS* directement comme des processus *Linux*, sans virtualisation complète, ce qui réduit la consommation de ressources.
- **Docker** : support des conteneurs pour des nœuds légers (serveurs *Linux*, outils de test réseau).
- **Linux bridges / veth / tap** : interconnexion des nœuds via des interfaces réseau virtuelles du noyau *Linux*.
- **UKSM (Ultra Kernel Samepage Merging)** : mécanisme de déduplication mémoire qui fusionne les pages *RAM* identiques entre *VMs*, réduisant considérablement la *RAM* consommée lorsque plusieurs instances du même OS tournent simultanément (disponible en édition *Pro*).

API et bibliothèques :

- **EVE-NG REST API** : API HTTP complète pour piloter EVE-NG par programmation (gestion des labs, démarrage/arrêt des nœuds, configuration des topologies). Les principaux endpoints sont `/api/labs`, `/api/nodes`, `/api/networks` et `/api/topologies`.
- **peve** : bibliothèque *Python* communautaire encapsulant l'API REST d'EVE-NG, facilitant l'automatisation des labs depuis des scripts *Python*.

Source : eve-ng.net - EVE-NG REST API : <https://www.eve-ng.net/index.php/documentation/howtos/how-to-eve-ng-api/>

Please check the website of ARISTA, a provider of SDN solutions. Try looking for libraries and extensions that will allow enriching the palette offered by natively by EVE.

Arista Networks est un fournisseur de solutions SDN dont le système d'exploitation **EOS (Extensible Operating System)** est conçu pour être entièrement programmable. Arista propose plusieurs bibliothèques et outils permettant d'enrichir les capacités d'EVE-NG.

Image virtuelle pour EVE-NG :

- **vEOS-lab** : image virtuelle d'EOS distribuée par Arista, compatible avec le moteur QEMU d'EVE-NG. Elle permet d'intégrer des switches/routeurs Arista réels dans les topologies EVE-NG pour tester les configurations EOS en lab.

Bibliothèques et extensions SDN :

- **eAPI** : API REST native d'EOS, exposée en HTTP/HTTPS sur chaque équipement Arista. Elle permet d'envoyer des commandes EOS et de récupérer l'état du réseau au format JSON, sans agent tiers.
- **pyeapi** : bibliothèque *Python* cliente pour eAPI, simplifiant l'automatisation des équipements Arista depuis des scripts *Python* (requêtes, configuration, parsing des résultats).
- **Arista AVD (Ansible Validated Designs)** : collection *Ansible* complète fournissant des rôles et *playbooks* pour concevoir, valider et déployer automatiquement des infrastructures réseau Arista. Publiée sur *Ansible Galaxy* et *GitHub*.
- **EOS SDK** : SDK C++ permettant de développer des agents personnalisés qui s'exécutent nativement sur EOS, avec accès direct aux tables de routage, aux interfaces et aux événements réseau.
- **CloudVision (CVP)** : plateforme centralisée de gestion et de télémétrie réseau, offrant du *streaming telemetry* en temps réel et un contrôle de configuration réseau à grande échelle.

Source : avd.arista.com - Arista AVD : <https://avd.arista.com/>

Source : pyeapi.readthedocs.io - pyeapi documentation : <https://pyeapi.readthedocs.io/>

Implémentation

La topologie réseau à implémenter est représentée par Figure 1.

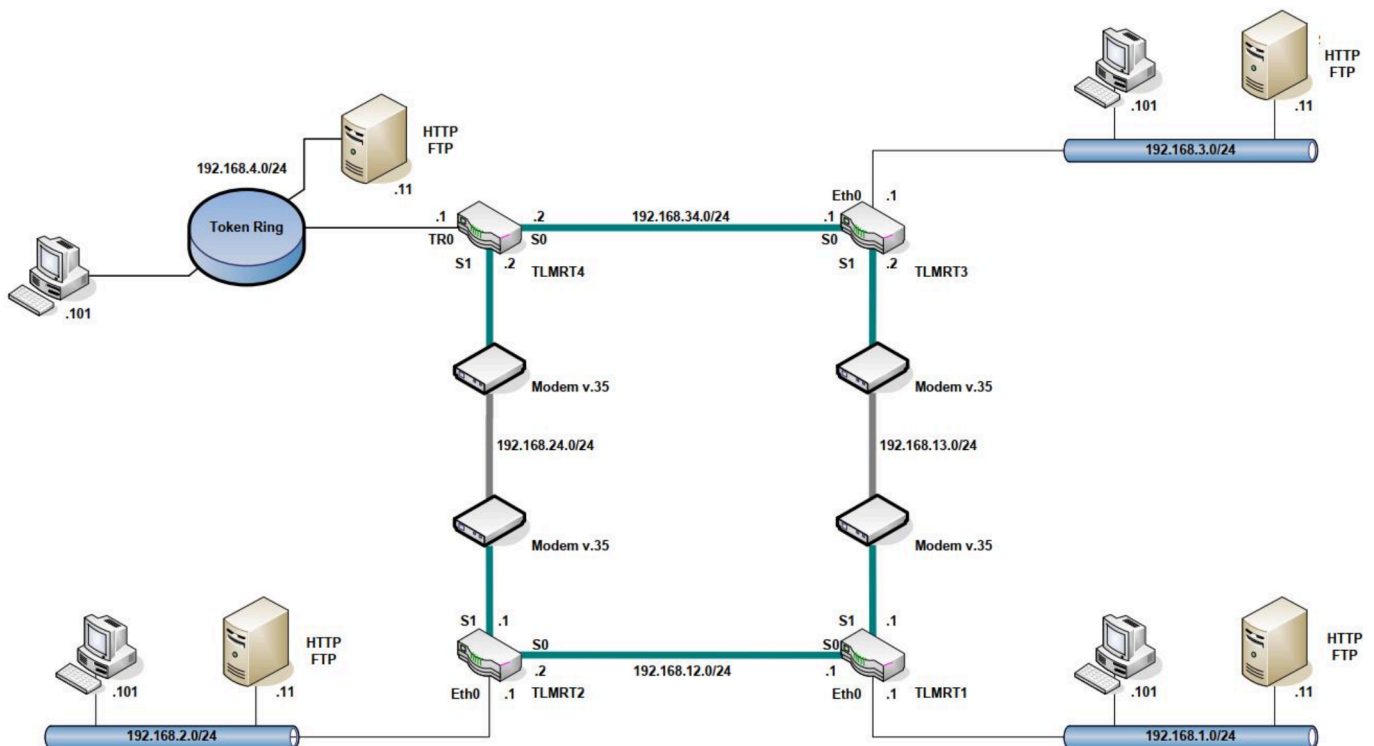


Figure 1: Topologie SDN déployée dans EVE-NG : 4 routeurs Arista vEOS (TLMRT1–4) interconnectés par 4 liens WAN, chaque routeur desservant un LAN avec 2 hôtes VPCS.

La topologie comporte quatre routeurs virtuels *Arista vEOS* (TLMRT1 à TLMRT4), chacun connecté à un réseau local (LAN1–LAN4, sous-réseaux 192.168.1.0/24 à 192.168.4.0/24). Chaque LAN héberge deux hôtes simulés par VPCS (un *Client* et un *Server*). Les routeurs sont reliés entre eux par quatre liens WAN point-à-point : WAN12, WAN13, WAN24 et WAN34. Le routage dynamique *OSPF* (aire 0) est configuré sur toutes les interfaces pour assurer la connectivité inter-LAN.

Script de déploiement

L'ensemble du déploiement de la topologie est automatisé via le script `deploy_topology.py`. Son fonctionnement se décompose en trois phases.

Phase 1 — Création du lab via l'API REST :

Le script s'authentifie auprès d'EVE-NG (`POST /api/auth/login`), supprime le lab existant si nécessaire, puis en crée un nouveau (`POST /api/labs`) pour obtenir un UUID unique. Ce lab est ensuite peuplé en construisant le fichier de topologie au format `.unl` (XML propriétaire d'EVE-NG) directement en mémoire.

Phase 2 — Construction et écriture du fichier `.unl` :

Le format `.unl` est un fichier XML décrivant la totalité de la topologie : réseaux virtuels (*bridges*), nœuds et câblage des interfaces. Le script construit cet XML en *Python* via `xml.etree.ElementTree`, y intègre les 8 réseaux, les 4 nœuds vEOS (type `qemu`, image `veos-4.36.0.1F`) et les 8 nœuds VPCS, avec leurs positions sur le *canvas* et leur câblage aux réseaux correspondants.

Le fichier résultant est déposé directement sur le serveur EVE-NG via *SFTP* :


```
1 sftp.open(f"/opt/unetlab/labs/{LAB_NAME}.unl", "wb").write(xml_bytes)
```

Problème rencontré avec les réseaux :

Une première approche consistait à créer les nœuds et les réseaux via les *endpoints REST* d'EVE-NG (POST /api/labs/{lab}/nodes, POST /api/labs/{lab}/networks). Cependant, EVE-NG CE ne dispose pas d'un *endpoint REST* permettant de câbler directement une interface d'un nœud à un réseau. Les connexions doivent être encodées dans le .unl. L'approche retenue est donc de construire l'intégralité du fichier .unl en *Python* et de le pousser via *SFTP*, court-circuitant ainsi les limitations de l'API REST pour le câblage.

Phase 3 – Déploiement des configurations de démarrage :

Les configurations initiales de chaque routeur (startup-config au format EOS) et de chaque hôte VPCS (startup.vpc) sont générées dynamiquement depuis les données de topology.py et déposées via *SFTP* dans les répertoires de nœuds d'EVE-NG :

```
1 /opt/unetlab/labs/{LAB_NAME}/{node_id}/startup-config # pour chaque routeur vEOS
2 /opt/unetlab/labs/{LAB_NAME}/{node_id}/startup.vpc   # pour chaque hôte VPCS
```

Le script se termine par une phase de vérification : il relit le .unl depuis le serveur et contrôle que chaque réseau, chaque routeur et chaque VPCS sont correctement câblés, en affichant le résultat pour chaque connexion.

Script d'exécution

Le script run_tests.py orchestre le démarrage du lab et la validation de la connectivité. Une fois les nœuds démarrés, les hôtes VPCS sont configurés avec leur adresse IP et leur passerelle. Les pings **intra-LAN** (entre les deux hôtes d'un même LAN) sont concluants : 4/4 passent systématiquement, ce qui confirme que le câblage virtuel EVE-NG est correct et que les nœuds VPCS communiquent bien au niveau 2.

```
1 [CONFIG] Configuring VPCS nodes...
2   [OK] Client1: 192.168.1.101/24 gw 192.168.1.1
3   [OK] Server1: 192.168.1.11/24  gw 192.168.1.1
4   [OK] Client2: 192.168.2.101/24 gw 192.168.2.1
5   [OK] Server2: 192.168.2.11/24  gw 192.168.2.1
6   [OK] Client3: 192.168.3.101/24 gw 192.168.3.1
7   [OK] Server3: 192.168.3.11/24  gw 192.168.3.1
8   [OK] Client4: 192.168.4.101/24 gw 192.168.4.1
9   [OK] Server4: 192.168.4.11/24  gw 192.168.4.1
10
11 [TEST] Same-LAN pings
12   [PASS] Client1 → Server1
13   [PASS] Client2 → Server2
14   [PASS] Client3 → Server3
15   [PASS] Client4 → Server4
16
17 4/4 passed
```

Les routeurs vEOS démarrent également : leurs sorties console sont visibles dans les *logs*, et chacun atteint le *prompt* login: après environ 3 500 secondes (émulation *TCG*, sans *KVM*). Cependant, nous n'arrivons pas à obtenir de convergence *OSPF*, et les pings **inter-LAN** échouent systématiquement : 0/12.

```
1 [CONFIG] Booting and configuring vEOS routers via telnet (parallel)...
2 [TLMRT1] waiting for login prompt (up to 7200s)...
3 [TLMRT1] still booting... (30s)
4 [TLMRT1] | Welcome to Arista Networks EOS 4.36.0.1F
5 ...
6 [TLMRT4] still booting... (3499s)
7 [TLMRT4] | Flash Memory size: 3.9G
8 [TLMRT4] ready (3527s)
9
10 [WAIT] Polling for gateway IPs (up to 1800s)...
11 [ 25s] 0/8 gateways up
12 [ 80s] 0/8 gateways up
13 ...
14 [1791s] 0/8 gateways up
15 [WARN] Only 0/8 gateways responded – agents may not have started
16
17 [TEST] Cross-LAN pings
18 [FAIL] Client1 → Server2
19 [FAIL] Client1 → Server3
20 [FAIL] Client1 → Server4
21 [FAIL] Client2 → Server3
22 [FAIL] Client2 → Server4
23 [FAIL] Client3 → Server4
24 [FAIL] Server1 → Client2
25 [FAIL] Server1 → Client3
26 [FAIL] Server1 → Client4
27 [FAIL] Server2 → Client3
28 [FAIL] Server2 → Client4
29 [FAIL] Server3 → Client4
30
31 0/12 passed
```

Les tentatives effectuées pour débloquer la situation sont les suivantes :

- **Injection de la startup-config via qemu-nbd** : avant de démarrer les nœuds, le script montait le disque QCOW2 de chaque routeur via *qemu-nbd* et écrivait le fichier *startup-config* directement sur la partition *flash* (partition 2, *ext4*). Le fichier était bien présent sur le disque (vérifié par remontage), mais les interfaces ne se levaient jamais après le *boot* : *ConfigAgent* ne semblait pas appliquer la configuration dans le délai observé.
- **Suppression des pokes ESC** : les envois périodiques du caractère *ESC* (`\x1b`) sur le port *Telnet*, destinés à accélérer le *boot* en sautant l'initialisation *EOS*, annulaient en réalité le processus *Aaa.sh* qui attendait le démarrage des agents (*wfw*). Retirer l'*ESC* a permis aux agents de démarrer correctement, mais n'a pas résolu le problème de configuration.
- **Augmentation du timeout de login** : le délai d'attente du *prompt* login: a été porté de 3 600 s à 7 200 s, car *wfw* s'exécute pendant 3 600 s à partir d'environ 400 s après le démarrage, reportant le login: effectif à 4 000 s. Cette correction a permis au script de ne plus *timeout* avant le *prompt*, mais n'a pas amélioré la configuration réseau.
- **Keepalive de la session EVE-NG** : pendant la fenêtre de *polling* des passerelles (jusqu'à 30 minutes), aucun appel à l'*API REST* n'était effectué. Un *cron* interne d'*EVE-NG* nettoyant les sessions *PHP* toutes les 30 minutes pouvait expirer la session et arrêter les nœuds silencieusement. Un appel périodique à `GET /api/labs/{lab}/nodes` toutes les 60 secondes a été ajouté pour maintenir la session active.

- **Configuration directe via Telnet CLI** : plutôt que de dépendre de *ConfigAgent*, le script se connecte en admin sur le port *Telnet* de chaque routeur après le login:, entre en mode configuration (configure terminal) et pousse les commandes d'interface et *OSPF* directement. Cette approche contourne entièrement le mécanisme de startup-config.

Malgré l'ensemble de ces tentatives, nous ne sommes pas parvenus à obtenir une convergence *OSPF* ni des pings inter-LAN fonctionnels (0/12).

Code source

topology.py

```
1  VEOS_QEMU = (  
2      "-machine type=pc-1.0,accel=tcg -serial mon:stdio -nographic "  
3      "-display none -no-user-config -rtc base=utc -boot order=d -cpu Haswell"  
4  )  
5  
6  ROUTERS = [  
7      {"name": "TLMRT1", "left": 600, "top": 400},  
8      {"name": "TLMRT2", "left": 200, "top": 400},  
9      {"name": "TLMRT3", "left": 600, "top": 150},  
10     {"name": "TLMRT4", "left": 200, "top": 150},  
11 ]  
12  
13 ROUTER_CONNECTIONS = {  
14     "TLMRT1": {1: "LAN1", 2: "WAN12", 3: "WAN13"},  
15     "TLMRT2": {1: "LAN2", 2: "WAN12", 3: "WAN24"},  
16     "TLMRT3": {1: "LAN3", 2: "WAN34", 3: "WAN13"},  
17     "TLMRT4": {1: "LAN4", 2: "WAN34", 3: "WAN24"},  
18 }  
19  
20 NETWORKS = [  
21     {"name": "LAN1", "left": 780, "top": 430},  
22     {"name": "LAN2", "left": 20, "top": 430},  
23     {"name": "LAN3", "left": 780, "top": 120},  
24     {"name": "LAN4", "left": 20, "top": 120},  
25     {"name": "WAN12", "left": 400, "top": 480},  
26     {"name": "WAN13", "left": 680, "top": 280},  
27     {"name": "WAN24", "left": 120, "top": 280},  
28     {"name": "WAN34", "left": 400, "top": 80},  
29 ]  
30  
31 VPCS_NODES = [  
32     {"name": "Client1", "lan": "LAN1", "left": 720, "top": 530},  
33     {"name": "Server1", "lan": "LAN1", "left": 840, "top": 530},  
34     {"name": "Client2", "lan": "LAN2", "left": 40, "top": 530},  
35     {"name": "Server2", "lan": "LAN2", "left": 160, "top": 530},  
36     {"name": "Client3", "lan": "LAN3", "left": 720, "top": 40},  
37     {"name": "Server3", "lan": "LAN3", "left": 840, "top": 40},  
38     {"name": "Client4", "lan": "LAN4", "left": 40, "top": 40},  
39     {"name": "Server4", "lan": "LAN4", "left": 160, "top": 40},  
40 ]  
41  
42 ROUTER_IFACE_CONFIGS = {  
43     "TLMRT1": [("Ethernet1", "192.168.1.1/24"), ("Ethernet2", "192.168.12.1/24"),  
44     ("Ethernet3", "192.168.13.1/24")],  
45     "TLMRT2": [("Ethernet1", "192.168.2.1/24"), ("Ethernet2", "192.168.12.2/24"),  
46     ("Ethernet3", "192.168.24.2/24")],
```

```

45     "TLMRT3": [("Ethernet1", "192.168.3.1/24"), ("Ethernet2", "192.168.34.3/24"),
46     ("Ethernet3", "192.168.13.3/24")],
47     "TLMRT4": [("Ethernet1", "192.168.4.1/24"), ("Ethernet2", "192.168.34.4/24"),
48     ("Ethernet3", "192.168.24.4/24")],
49 }
50 VPCS_IP_CONFIGS = {
51     "Client1": ("192.168.1.101/24", "192.168.1.1"),
52     "Server1": ("192.168.1.11/24", "192.168.1.1"),
53     "Client2": ("192.168.2.101/24", "192.168.2.1"),
54     "Server2": ("192.168.2.11/24", "192.168.2.1"),
55     "Client3": ("192.168.3.101/24", "192.168.3.1"),
56     "Server3": ("192.168.3.11/24", "192.168.3.1"),
57     "Client4": ("192.168.4.101/24", "192.168.4.1"),
58     "Server4": ("192.168.4.11/24", "192.168.4.1"),
59 }
60 VPCS_IPS = {name: cidr.split("/")[0] for name, (cidr, _) in VPCS_IP_CONFIGS.items()}
61
62 SAME_LAN_PAIRS = [
63     ("Client1", "Server1"),
64     ("Client2", "Server2"),
65     ("Client3", "Server3"),
66     ("Client4", "Server4"),
67 ]
68
69 TEST_PAIRS = [
70     ("Client1", "Server2"),
71     ("Client1", "Server3"),
72     ("Client1", "Server4"),
73     ("Client2", "Server3"),
74     ("Client2", "Server4"),
75     ("Client3", "Server4"),
76     ("Server1", "Client2"),
77     ("Server1", "Client3"),
78     ("Server1", "Client4"),
79     ("Server2", "Client3"),
80     ("Server2", "Client4"),
81     ("Server3", "Client4"),
82 ]

```

deploy_topology.py

```
1  #!/usr/bin/env python3
2
3  import os
4  import sys
5  import time
6  import uuid
7  import ipaddress
8  import xml.etree.ElementTree as ET
9  from pathlib import Path
10 import paramiko
11 import requests
12 from urllib.parse import quote
13 from dotenv import load_dotenv
14
15 load_dotenv(Path(__file__).parent / ".env")
16
17 # ----- Constants
18
19 EVE_URL      = os.getenv("EVE_URL",      "http://192.168.1.113")
20 USERNAME     = os.getenv("USERNAME",     "admin")
21 PASSWORD     = os.getenv("PASSWORD",     "eve")
22 SSH_USER     = os.getenv("SSH_USER",     "root")
23 SSH_PASS     = os.getenv("SSH_PASS",     "eve")
24 LAB_NAME     = os.getenv("LAB_NAME",     "Test1")
25 LAB_FOLDER   = os.getenv("LAB_FOLDER",   "/")
26 VEOS_IMAGE   = os.getenv("VEOS_IMAGE",   "veos-4.36.0.1F")
27
28 # ----- Topology (edit topology.py to change names, IPs, or canvas positions)
29
30 from topology import (VEOS_QEMU, ROUTERS, ROUTER_CONNECTIONS, NETWORKS,
31                       VPCS_NODES, ROUTER_IFACE_CONFIGS, VPCS_IP_CONFIGS)
32
33 # ----- Helpers
34
35 def lab_url_path(name, folder):
36     folder = folder.strip("/")
37     if folder:
38         return quote(f"{folder}/{name}.unl", safe="")
39     return f"{name}.unl"
40
41 def check(r, action):
42     if r.status_code not in (200, 201):
43         print(f"[ERROR] {action} → HTTP {r.status_code}: {r.text}")
44         sys.exit(1)
45     data = r.json()
46     if data.get("code") not in (200, 201):
47         print(f"[ERROR] {action} → {data.get('message', data)}")
48         sys.exit(1)
49     return data
50
51 def make_router_config(name, ifaces):
52     lines = [f"hostname {name}", "!", "ip routing", "!"]
53     nets = []
54     for iface, addr in ifaces:
55         lines += [f"interface {iface}", f"    ip address {addr}", "    no shutdown", "!"]
56         nets.append(str(ipaddress.ip_interface(addr).network))
57     lines.append("router ospf 1")
58     for net in nets:
59         lines.append(f"    network {net} area 0.0.0.0")
60     lines += ["!", "end", ""]
61     return "\n".join(lines)
62
63 # ----- Deploy
64
65 def main():
```

```

66 session = requests.Session()
67 session.headers.update({"Content-Type": "application/json"})
68
69 # Login
70 r = session.post(f"{EVE_URL}/api/auth/login",
71                 json={"username": USERNAME, "password": PASSWORD, "html5": "-1"})
72 check(r, "login")
73 print("[OK] Authenticated")
74
75 # Delete existing lab
76 lab_path = lab_url_path(LAB_NAME, LAB_FOLDER)
77 r = session.delete(f"{EVE_URL}/api/labs/{lab_path}")
78 if r.status_code == 200:
79     print(f"[OK] Existing lab '{LAB_NAME}' deleted, waiting for cleanup...")
80     for _ in range(30):
81         time.sleep(1)
82         r = session.get(f"{EVE_URL}/api/labs/{lab_path}")
83         if r.status_code == 404 or r.json().get("code") == 404:
84             break
85     time.sleep(2)
86     print("[OK] Lab fully removed")
87
88 # Create lab (generates UUID and empty .unl on disk)
89 r = session.post(f"{EVE_URL}/api/labs", json={
90     "name": LAB_NAME,
91     "path": LAB_FOLDER,
92     "version": "1",
93     "description": "SDN Lab – Arista vEOS ring topology",
94 })
95 check(r, "Lab creation")
96 print(f"[OK] Lab '{LAB_NAME}' created")
97
98 # Read UUID from the generated .unl
99 base = f"{EVE_URL}/api/labs/{lab_url_path(LAB_NAME, LAB_FOLDER)}"
100 lab_uuid = check(session.get(base), "get lab")["data"]["id"]
101 print(f"[OK] Lab UUID: {lab_uuid}")
102
103 # ----- Build complete .unl XML
104
105 net_ids = {net["name"]: i for i, net in enumerate(NETWORKS, start=1)}
106
107 lab_el = ET.Element("lab",
108     name=LAB_NAME, id=lab_uuid,
109     version="1", scripttimeout="600", lock="0",
110 )
111 ET.SubElement(lab_el, "description").text = "SDN Lab – Arista vEOS ring topology"
112 topology = ET.SubElement(lab_el, "topology")
113 nodes_el = ET.SubElement(topology, "nodes")
114 nets_el = ET.SubElement(topology, "networks")
115
116 # Networks
117 for net in NETWORKS:
118     ET.SubElement(nets_el, "network",
119         id=str(net_ids[net["name"]]), type="bridge",
120         name=net["name"], left=str(net["left"]), top=str(net["top"]),
121         visibility="1", icon="lan.png",
122     )
123     print(f"[OK] network {net['name']:6s} → id={net_ids[net['name']]}")
124
125 # Routers (ids 1-4)
126 for i, router in enumerate(ROUTERS, start=1):
127     node_el = ET.SubElement(nodes_el, "node",
128         id=str(i), name=router["name"],
129         type="qemu", template="veos", image=VEOS_IMAGE,
130         console="telnet", cpu="2", cpulimit="1", ram="2048", ethernet="4",
131         uuid=str(uuid.uuid4()),

```

```

132         qemu_options=VEOS_QEMU, qemu_version="2.4.0", qemu_arch="x86_64",
133         delay="0", icon="Router.png", config="1",
134         left=str(router["left"]), top=str(router["top"]),
135     )
136     for iface_idx, net_name in ROUTER_CONNECTIONS[router["name"]].items():
137         ET.SubElement(node_el, "interface",
138             id=str(iface_idx), name=f"Eth{iface_idx}",
139             type="ethernet", network_id=str(net_ids[net_name]),
140         )
141     print(f"[OK] Router {router['name']:6s} → id={i}")
142
143 # VPCS (ids 5-12)
144 for i, vpc in enumerate(VPCS_NODES, start=len(ROUTERS) + 1):
145     node_el = ET.SubElement(nodes_el, "node",
146         id=str(i), name=vpc["name"],
147         type="vpcs", template="vpcs", image="",
148         ethernet="1", delay="0", icon="Router.png", config="1",
149         left=str(vpc["left"]), top=str(vpc["top"]),
150     )
151     ET.SubElement(node_el, "interface",
152         id="0", name="eth0",
153         type="ethernet", network_id=str(net_ids[vpc["lan"]]),
154     )
155     print(f"[OK] VPCS {vpc['name']:8s} → id={i}, wired to {vpc['lan']}")
156
157 # ----- Write .unl via SSH/SFTP
158 ET.indent(lab_el)
159 xml_bytes = (
160     b'<?xml version="1.0" encoding="UTF-8" standalone="yes"?>\n'
161     + ET.tostring(lab_el, encoding="unicode").encode()
162 )
163
164 ssh = paramiko.SSHClient()
165 ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
166 ssh.connect(EVE_URL.split("/")[1], username=SSH_USER, password=SSH_PASS)
167 sftp = ssh.open_sftp()
168 with sftp.open(f"/opt/unetlab/labs/{LAB_NAME}.unl", "wb") as f:
169     f.write(xml_bytes)
170 sftp.close()
171 ssh.close()
172 print("[OK] .unl written via SFTP")
173
174 # ----- Push startup configs via SSH
175 print("\n[CONFIGS] Pushing startup configurations...")
176 ssh_cfg = paramiko.SSHClient()
177 ssh_cfg.set_missing_host_key_policy(paramiko.AutoAddPolicy())
178 ssh_cfg.connect(EVE_URL.split("/")[1], username=SSH_USER, password=SSH_PASS)
179 sftp_cfg = ssh_cfg.open_sftp()
180 config_base = f"/opt/unetlab/labs/{LAB_NAME}"
181
182 for i, router in enumerate(ROUTERS, start=1):
183     node_dir = f"{config_base}/{i}"
184     _, stdout, _ = ssh_cfg.exec_command(f"mkdir -p {node_dir}")
185     stdout.channel.recv_exit_status()
186     cfg = make_router_config(router["name"], ROUTER_IFACE_CONFIGS[router["name"]])
187     with sftp_cfg.open(f"{node_dir}/startup-config", "w") as f:
188         f.write(cfg)
189     print(f"[OK] Router {router['name']:6s} startup-config written")
190
191 for i, vpc in enumerate(VPCS_NODES, start=len(ROUTERS) + 1):
192     node_dir = f"{config_base}/{i}"
193     _, stdout, _ = ssh_cfg.exec_command(f"mkdir -p {node_dir}")
194     stdout.channel.recv_exit_status()
195     ip_cidr, gw = VPCS_IP_CONFIGS[vpc["name"]]
196     with sftp_cfg.open(f"{node_dir}/startup.vpc", "w") as f:

```

```

197         f.write(f"ip {ip_cidr} {gw}\n")
198         print(f"[OK] VPCS {vpc['name']:8s} startup.vpc written")
199
200     sftp_cfg.close()
201     ssh_cfg.close()
202
203     # ---- Verify
204     print("\n[VERIFY] Reading back .unl for verification...")
205     ssh2 = paramiko.SSHClient()
206     ssh2.set_missing_host_key_policy(paramiko.AutoAddPolicy())
207     ssh2.connect(EVE_URL.split("//")[1], username=SSH_USER, password=SSH_PASS)
208     sftp2 = ssh2.open_sftp()
209     with sftp2.open(f"/opt/unetlab/labs/{LAB_NAME}.unl", "rb") as f:
210         written = ET.fromstring(f.read())
211     sftp2.close()
212     ssh2.close()
213
214     errors = 0
215
216     # Check networks
217     found_nets = {el.get("name"): int(el.get("id")) for el in written.findall(".//
network")}
218     for net in NETWORKS:
219         if net["name"] not in found_nets:
220             print(f" [FAIL] network {net['name']} missing")
221             errors += 1
222         elif found_nets[net["name"]] != net_ids[net["name"]]:
223             print(f" [FAIL] network {net['name']} wrong id={found_nets[net['name']]}"
224             errors += 1
225         else:
226             print(f" [OK]   network {net['name']:6s} id={found_nets[net['name']]}"
227
228     # Check router interface wiring
229     for i, router in enumerate(ROUTERS, start=1):
230         node_el = written.find(f".//node[@id='{i}']")
231         if node_el is None:
232             print(f" [FAIL] router {router['name']} missing")
233             errors += 1
234             continue
235         for iface_idx, net_name in ROUTER_CONNECTIONS[router["name"]].items():
236             iface_el = node_el.find(f"interface[@id='{iface_idx}']")
237             expected = str(net_ids[net_name])
238             if iface_el is None:
239                 print(f" [FAIL] {router['name']} Eth{iface_idx} interface missing")
240                 errors += 1
241             elif iface_el.get("network_id") != expected:
242                 print(f" [FAIL] {router['name']} Eth{iface_idx} → net
{iface_el.get('network_id')} (expected {expected})"
243                 errors += 1
244             else:
245                 print(f" [OK]   {router['name']} Eth{iface_idx} → {net_name}
(net_id={expected})"
246
247     # Check VPCS wiring
248     for i, vpc in enumerate(VPCS_NODES, start=len(ROUTERS) + 1):
249         node_el = written.find(f".//node[@id='{i}']")
250         if node_el is None:
251             print(f" [FAIL] VPCS {vpc['name']} missing")
252             errors += 1
253             continue
254         iface_el = node_el.find("interface[@id='0']")
255         expected = str(net_ids[vpc["lan"]])
256         if iface_el is None:
257             print(f" [FAIL] {vpc['name']} interface missing")
258             errors += 1

```



```

259         elif iface_el.get("network_id") != expected:
260             print(f" [FAIL] {vpc['name']} → net {iface_el.get('network_id')} (expected
{expected})")
261             errors += 1
262         else:
263             print(f" [OK] {vpc['name']:8s} → {vpc['lan']} (net_id={expected})")
264
265     if errors == 0:
266         print(f"\n[DONE] Topology deployed and verified → {EVE_URL}")
267     else:
268         print(f"\n[DONE] Deployed with {errors} verification error(s) → {EVE_URL}")
269
270 # ----- Main
271
272 if __name__ == "__main__":
273     main()

```

run_topology.py

```

1  #!/usr/bin/env python3
2
3  import os
4  import sys
5  import time
6  import socket
7  import threading
8  import ipaddress
9  import paramiko
10 import requests
11 from pathlib import Path
12 from urllib.parse import quote, urlparse
13 from dotenv import load_dotenv
14 from topology import VPCS_IP_CONFIGS, VPCS_IPS, SAME_LAN_PAIRS, TEST_PAIRS,
ROUTER_IFACE_CONFIGS
15
16 load_dotenv(Path(__file__).parent / ".env")
17
18 EVE_URL = os.getenv("EVE_URL", "http://192.168.1.113")
19 EVE_HOST = urlparse(EVE_URL).hostname
20 USERNAME = os.getenv("USERNAME", "admin")
21 PASSWORD = os.getenv("PASSWORD", "eve")
22 SSH_USER = os.getenv("SSH_USER", "root")
23 SSH_PASS = os.getenv("SSH_PASS", "eve")
24 LAB_NAME = os.getenv("LAB_NAME", "Test1")
25 LAB_FOLDER = os.getenv("LAB_FOLDER", "/")
26
27 VEOS_BOOT_TIMEOUT = 900 # seconds to wait for EVE-NG status=2
28 VEOS_LOGIN_TIMEOUT = 7200 # seconds to wait for vEOS login prompt (TCG is slow; wfw runs
for 3600s starting ~400s in)
29
30 # --- telnet helpers ---
31
32 IAC = 255
33 WILL = 251
34 WONT = 252
35 DO = 253
36 DONT = 254
37
38 def _negotiate(sock, data):
39     out = bytearray()
40     i = 0
41     while i < len(data):
42         b = data[i]
43         if b == IAC and i + 2 < len(data):

```

```

44         cmd, opt = data[i+1], data[i+2]
45         if cmd == WILL:
46             sock.send(bytes([IAC, DONT, opt]))
47         elif cmd == DO:
48             sock.send(bytes([IAC, WONT, opt]))
49         i += 3
50     else:
51         out.append(b)
52         i += 1
53     return bytes(out)
54
55 def _read_until(sock, prompt, timeout=15):
56     buf = b""
57     deadline = time.time() + timeout
58     while time.time() < deadline:
59         sock.settimeout(max(0.1, deadline - time.time()))
60         try:
61             chunk = sock.recv(4096)
62         except socket.timeout:
63             break
64         if not chunk:
65             break
66         buf += _negotiate(sock, chunk)
67         if prompt in buf:
68             break
69     return buf
70
71 def _read_until_any(sock, prompts, timeout=15):
72     buf = b""
73     deadline = time.time() + timeout
74     while time.time() < deadline:
75         sock.settimeout(max(0.1, deadline - time.time()))
76         try:
77             chunk = sock.recv(4096)
78         except socket.timeout:
79             break
80         if not chunk:
81             break
82         buf += _negotiate(sock, chunk)
83         if any(p in buf for p in prompts):
84             break
85     return buf
86
87 def vpcs_run(port, commands, timeout=15):
88     PROMPT = b"VPCS> "
89     sock = socket.create_connection((EVE_HOST, port), timeout=timeout)
90     _read_until(sock, PROMPT, timeout)
91     # flush any stale buffered output before issuing commands
92     sock.sendall(b"\n")
93     _read_until(sock, PROMPT, timeout)
94     results = []
95     for cmd in commands:
96         sock.sendall(cmd.encode() + b"\n")
97         results.append(_read_until(sock, PROMPT, timeout).decode(errors="replace"))
98     sock.close()
99     return results
100
101 def boot_and_configure_veos(port, router_name):
102     """Wait for vEOS login prompt, log in as admin, and apply interface+OSPF config via
103     CLI."""
104     iface_configs = ROUTER_IFACE_CONFIGS[router_name]
105     config_cmds = ["ip routing"]
106     for iface, ip_cidr in iface_configs:

```

```

107     config_cmds += [f"interface {iface}", f"    ip address {ip_cidr}", "    no
shutdown"]
108     config_cmds.append("router ospf 1")
109     for _, ip_cidr in iface_configs:
110         net = str(ipaddress.ip_interface(ip_cidr).network)
111         config_cmds.append(f"    network {net} area 0.0.0.0")
112
113     print(f"    [{router_name}] waiting for login prompt (up to {VEOS_LOGIN_TIMEOUT}
s)...", flush=True)
114     start = time.time()
115     deadline = start + VEOS_LOGIN_TIMEOUT
116     _line_buf = [""]
117
118     try:
119         sock = socket.create_connection((EVE_HOST, port), timeout=30)
120     except Exception as e:
121         print(f"    [{router_name}] connection failed: {e}", flush=True)
122         return False
123
124     sock.sendall(b"\n")
125     buf = b""
126
127     while time.time() < deadline:
128         sock.settimeout(30)
129         try:
130             chunk = sock.recv(4096)
131         except socket.timeout:
132             elapsed = int(time.time() - start)
133             print(f"    [{router_name}] still booting... ({elapsed}s)", flush=True)
134             sock.sendall(b"\n")
135             continue
136         if not chunk:
137             break
138         negotiated = _negotiate(sock, chunk)
139         buf += negotiated
140         for char in negotiated.decode(errors="replace"):
141             if char == "\n":
142                 if _line_buf[0].strip():
143                     print(f"    [{router_name}] | {_line_buf[0]}", flush=True)
144                     _line_buf[0] = ""
145             elif char != "\r":
146                 _line_buf[0] += char
147         if b"login:" in buf:
148             break
149     else:
150         sock.close()
151         print(f"    [{router_name}] login prompt not reached within {VEOS_LOGIN_TIMEOUT}
s", flush=True)
152         return False
153
154     if b"login:" not in buf:
155         sock.close()
156         print(f"    [{router_name}] login prompt not reached (EOF)", flush=True)
157         return False
158
159     elapsed = int(time.time() - start)
160     print(f"    [{router_name}] reached login: ({elapsed}s), configuring...", flush=True)
161     time.sleep(1)
162     sock.sendall(b"admin\r\n")
163
164     buf = _read_until_any(sock, [b"Password:", b">", b"#"], timeout=30)
165     if b"Password:" in buf:
166         sock.sendall(b"\r\n")
167         buf = _read_until_any(sock, [b">", b"#"], timeout=30)
168     if not (b">" in buf or b"#" in buf):

```

```

169     sock.close()
170     print(f"    [{router_name}] no shell prompt after login", flush=True)
171     return False
172
173     if b">" in buf and b"#" not in buf:
174         sock.sendall(b"enable\r\n")
175         buf = _read_until(sock, b"#", timeout=30)
176         if b"#" not in buf:
177             sock.close()
178             print(f"    [{router_name}] enable failed", flush=True)
179             return False
180
181     sock.sendall(b"configure terminal\r\n")
182     buf = _read_until(sock, b"(config)", timeout=30)
183     if b"(config)" not in buf:
184         sock.close()
185         print(f"    [{router_name}] configure terminal failed", flush=True)
186         return False
187
188     for cmd in config_cmds:
189         sock.sendall(cmd.encode() + b"\r\n")
190         time.sleep(1.0)
191
192     sock.sendall(b"end\r\n")
193     _read_until(sock, b"#", timeout=30)
194     sock.sendall(b"write memory\r\n")
195     _read_until(sock, b"#", timeout=30)
196
197     # Verify interfaces are up
198     sock.sendall(b"show ip interface brief\r\n")
199     ifbuf = _read_until(sock, b"#", timeout=30)
200     for line in ifbuf.decode(errors="replace").splitlines():
201         if line.strip():
202             print(f"    [{router_name}] | {line}", flush=True)
203
204     sock.close()
205     elapsed = int(time.time() - start)
206     print(f"    [{router_name}] configured and saved ({elapsed}s)", flush=True)
207     return True
208
209 # --- EVE-NG helpers ---
210
211 def lab_url_path(name, folder):
212     folder = folder.strip("/")
213     if folder:
214         return quote(f"{folder}/{name}.unl", safe="")
215     return f"{name}.unl"
216
217 def check(r, action):
218     if r.status_code not in (200, 201):
219         print(f"[ERROR] {action} → HTTP {r.status_code}: {r.text}")
220         sys.exit(1)
221     data = r.json()
222     if data.get("code") not in (200, 201):
223         print(f"[ERROR] {action} → {data.get('message', data)}")
224         sys.exit(1)
225     return data
226
227 def telnet_port(node_info):
228     return urlparse(node_info.get("url", "")).port
229
230 def run_pings(pairs, nodes, label):
231     print(f"\n[TEST] {label}")
232     passed = 0
233     for src, dst in pairs:
234         src_info = nodes.get(src)
235         if not src_info:

```

```

236         print(f" [FAIL] {src} → {dst} (node not found)")
237         continue
238     port = telnet_port(src_info)
239     target = VPCS_IPS[dst]
240     try:
241         outs = vpcs_run(port, [f"ping {target}"])
242         ok = "84 bytes" in outs[0]
243         print(f" {'[PASS]'} if ok else '[FAIL]'} {src} → {dst}")
244         if ok:
245             passed += 1
246     except Exception as e:
247         print(f" [FAIL] {src} → {dst} ({e})")
248     print(f"\n{passed}/{len(pairs)} passed")
249     return passed
250
251 def main():
252     session = requests.Session()
253     session.headers.update({"Content-Type": "application/json"})
254
255     # Authenticate
256     r = session.post(f"{EVE_URL}/api/auth/login",
257                     json={"username": USERNAME, "password": PASSWORD, "html5": "-1"})
258     check(r, "login")
259     print("[OK] Authenticated")
260
261     # Open lab
262     lab_path = lab_url_path(LAB_NAME, LAB_FOLDER)
263     base = f"{EVE_URL}/api/labs/{lab_path}"
264     lab_data = check(session.get(base), "open lab")
265     lab_uuid = lab_data["data"]["id"]
266     print(f"[OK] Lab '{LAB_NAME}' opened (uuid={lab_uuid})")
267
268     # Start all nodes
269     try:
270         r = session.get(f"{base}/nodes/start", timeout=10)
271         if r.status_code == 200 and r.json().get("code") in (200, 201):
272             print("[OK] All nodes started")
273         else:
274             print(f"[WARN] nodes/start → HTTP {r.status_code} / {r.text[:80]}")
275     except requests.exceptions.Timeout:
276         print("[OK] nodes/start sent (response timed out, nodes booting)")
277
278     # Wait for all nodes to reach status=2
279     print(f"\n[WAIT] Waiting for all nodes (timeout {VEOS_BOOT_TIMEOUT}s)...")
280     data = check(session.get(f"{base}/nodes"), "get nodes")
281     node_ids = list(data["data"].keys())
282
283     pending = set(node_ids)
284     deadline = time.time() + VEOS_BOOT_TIMEOUT
285     while pending and time.time() < deadline:
286         statuses = {}
287         for nid in list(pending):
288             r = session.get(f"{base}/nodes/{nid}")
289             if r.status_code == 200:
290                 info = r.json().get("data", {})
291                 status = info.get("status", 0)
292                 name = info.get("name", nid)
293                 statuses[name] = status
294                 if status in (2, "2"):
295                     pending.discard(nid)
296                     print(f" [UP] {name} (id={nid})")
297         if pending:
298             summary = ", ".join(f"{n}={s}" for n, s in sorted(statuses.items()))
299             elapsed = int(VEOS_BOOT_TIMEOUT - (deadline - time.time()))
300             print(f" [{elapsed:3d}s] still pending: {summary}")

```

```

301         time.sleep(10)
302
303     if pending:
304         print(f"[WARN] Still not running after timeout: ids={pending}")
305     else:
306         print("[OK] All nodes are running")
307
308     # Build name -> node info map (preserve node_id for QCOW2 injection)
309     data = check(session.get(f"{base}/nodes"), "get nodes")
310     nodes = {}
311     for nid, info in data["data"].items():
312         info["node_id"] = nid
313         nodes[info["name"]] = info
314
315     # Configure all VPCS nodes (retry up to 3 times on failure)
316     print("\n[CONFIG] Configuring VPCS nodes...")
317     for name in VPCS_IP_CONFIGS:
318         info = nodes.get(name)
319         if not info:
320             print(f" [WARN] {name} not found in lab")
321             continue
322         port = telnet_port(info)
323         if not port:
324             print(f" [WARN] {name} has no telnet port")
325             continue
326         ip_cidr, gw = VPCS_IP_CONFIGS[name]
327         for attempt in range(1, 4):
328             try:
329                 outs = vpcs_run(port, [f"ip {ip_cidr} {gw}"])
330                 ok = "VPCS :" in outs[0]
331             except Exception as e:
332                 ok = False
333                 outs = [str(e)]
334             if ok:
335                 print(f" [OK] {name}: {ip_cidr} gw {gw}" + (f" (attempt {attempt})" if
attempt > 1 else ""))
336                 break
337             if attempt < 3:
338                 time.sleep(2)
339         else:
340             print(f" [FAIL] {name}: {ip_cidr} gw {gw} (3 attempts failed)")
341             print(f"    → {outs[0].strip()}")
342
343     # Same-LAN pings (no routing needed)
344     run_pings(SAME_LAN_PAIRS, nodes, "Same-LAN pings")
345
346     # Boot vEOS routers and configure via telnet CLI after login: – in parallel
347     print("\n[CONFIG] Booting and configuring vEOS routers via telnet (parallel)...")
348     router_ports = {}
349     for router_name in ROUTER_IFACE_CONFIGS:
350         info = nodes.get(router_name)
351         if not info:
352             print(f" [WARN] {router_name} not found")
353             continue
354         port = telnet_port(info)
355         if port:
356             router_ports[router_name] = port
357         else:
358             print(f" [WARN] {router_name} has no telnet port")
359
360     results = {}
361     threads = []
362     for router_name, port in router_ports.items():
363         def _wait(name=router_name, p=port):
364             results[name] = boot_and_configure_veos(p, name)

```

```

365     t = threading.Thread(target=_wait, daemon=True)
366     threads.append(t)
367     t.start()
368     for t in threads:
369         t.join()
370     routers_ok = all(results.get(n, False) for n in router_ports)
371
372     # Poll for gateways to respond – agents apply startup-config in background after
login:
373     # Under TCG, this can take several minutes after the login prompt appears.
374     gw_pairs = [(name, VPCS_IP_CONFIGS[name][1]) for name in VPCS_IP_CONFIGS]
375     GW_POLL_TIMEOUT = 1800 # 30 minutes
376     print(f"\n[WAIT] Polling for gateway IPs (up to {GW_POLL_TIMEOUT}s, agents start in
background)...")
377     gw_deadline = time.time() + GW_POLL_TIMEOUT
378     gw_passed = 0
379     gw_api_last = time.time()
380     while time.time() < gw_deadline:
381         # keepalive: touch the EVE-NG session every 60s to prevent PHP session expiry
382         if time.time() - gw_api_last > 60:
383             try:
384                 session.get(f"{base}/nodes", timeout=10)
385             except Exception:
386                 pass
387             gw_api_last = time.time()
388             gw_passed = 0
389             for vpcs_name, gw_ip in gw_pairs:
390                 info = nodes.get(vpcs_name)
391                 if not info:
392                     continue
393                 try:
394                     outs = vpcs_run(telnet_port(info), [f"ping {gw_ip}"])
395                     if "84 bytes" in outs[0]:
396                         gw_passed += 1
397                 except Exception:
398                     pass
399             elapsed = int(GW_POLL_TIMEOUT - (gw_deadline - time.time()))
400             print(f" [{elapsed:3d}s] {gw_passed}/{len(gw_pairs)} gateways up", flush=True)
401             if gw_passed == len(gw_pairs):
402                 print("[OK] All gateways up – EOS agents applied startup-config")
403                 break
404             time.sleep(30)
405         else:
406             print(f"[WARN] Only {gw_passed}/{len(gw_pairs)} gateways responded – agents may
not have started")
407
408     # Wait for OSPF convergence after interfaces are up
409     if gw_passed > 0:
410         print("\n[WAIT] Waiting 90s for OSPF to converge...")
411         time.sleep(90)
412
413     # Cross-LAN pings (require routing)
414     run_pings(TEST_PAIRS, nodes, "Cross-LAN pings")
415
416     # Stop all nodes
417     try:
418         session.get(f"{base}/nodes/stop", timeout=10)
419         print("\n[OK] All nodes stopped")
420     except requests.exceptions.Timeout:
421         print("\n[OK] nodes/stop sent")
422
423     sys.exit(0)
424
425 if __name__ == "__main__":
426     main()

```

Conclusion

Ce laboratoire a permis de mettre en place un environnement d'émulation réseau complet basé sur *EVE-NG* et des routeurs *Arista vEOS*, entièrement déployé par des scripts *Python* automatisés. La topologie en anneau de quatre routeurs avec ses huit hôtes *VPCS* a été construite et câblée de manière programmatique via l'*API REST* et *SFTP*. Les tests de connectivité intra-*LAN* ont été concluants (4/4 pings réussis), ce qui confirme la validité du câblage virtuel et de la configuration des hôtes. En revanche, la convergence *OSPF* n'a pas pu être obtenue, et les pings inter-*LAN* ont échoué dans leur intégralité (0/12). La principale limitation identifiée réside dans la lenteur extrême de l'émulation *TCG* (sans *KVM*), qui rend l'application de la configuration *EOS* difficile à orchestrer de manière fiable.

Ce travail met en lumière les contraintes pratiques liées à l'utilisation d'*EVE-NG Community* sans accélération matérielle : des temps de démarrage proches d'une heure par routeur et une *API REST* incomplète pour le câblage des interfaces. La comparaison avec *GNS3* suggère que ce dernier offrirait une expérience plus fluide pour des labs mono-utilisateur nécessitant une interaction rapide. Sur le plan méthodologique, l'approche d'automatisation adoptée — construction du fichier `.unl` en mémoire et déploiement via *SFTP*, s'est révélée robuste et reproductible. Les outils *Arista* tels que `pyeapi` et `eAPI` restent des pistes prometteuses pour pousser la configuration sans dépendre du mécanisme de `startup-config` d'*EVE-NG*. Ce laboratoire constitue une introduction solide aux enjeux d'automatisation et de programmabilité des réseaux dans un contexte *SDN* réel.